

Bloxorz 3D Report

Computer Graphics Course Project

Submission date: June 11, 2026

Introduction

Bloxorz 3D is a browser-based 3D puzzle game inspired by the classic Bloxorz block-rolling mechanic. The player controls a rectangular block on a tile board and tries to reach the goal tile. The goal is accepted only when the full block is standing upright on it, so the puzzle is about both position and orientation.

The project is implemented as a real-time WebGL application with Three.js. It is not only a puzzle logic demo: the final version includes a complete menu flow, level selection, eight playable levels, saved progress, animated block movement, level mechanics, a custom editor, procedural audio, visual feedback, postprocessing, and an automated level verification script.

The main design decision is simple: the game state is discrete, but the visual output is continuous. The board logic uses exact grid cells, while the rendered block uses smooth 3D transforms, lighting, shadows, particles, and camera movement.

Motivation

The motivation was to build a project that clearly belongs to Computer Graphics while still being playable. A Bloxorz-style game is useful for this because every move can be explained with transformations:

- The block translates and rotates in 3D.

- The camera follows the active object using a view transform.
- The perspective camera projects the 3D world to the screen.
- Lighting, materials, shadows, fog, and postprocessing affect the final frame.
- User input changes a state graph, then the scene animates toward that state.

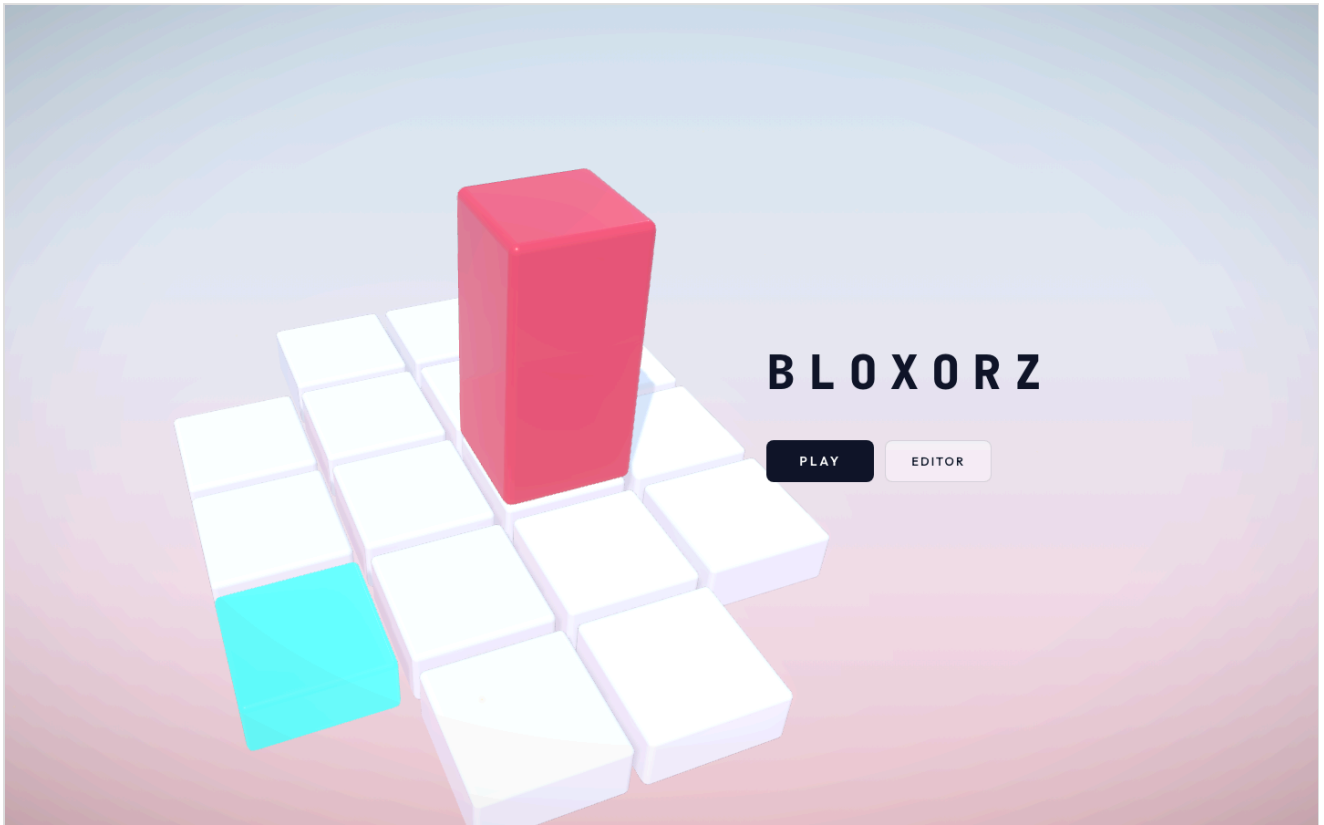
The game also gives a natural place to use algorithms. The hint system uses breadth-first search, and the same idea is used to verify the official par values. That makes the project stronger than a scene-only graphics demo, because the implementation is both graphical and rule-driven.

What Was Built

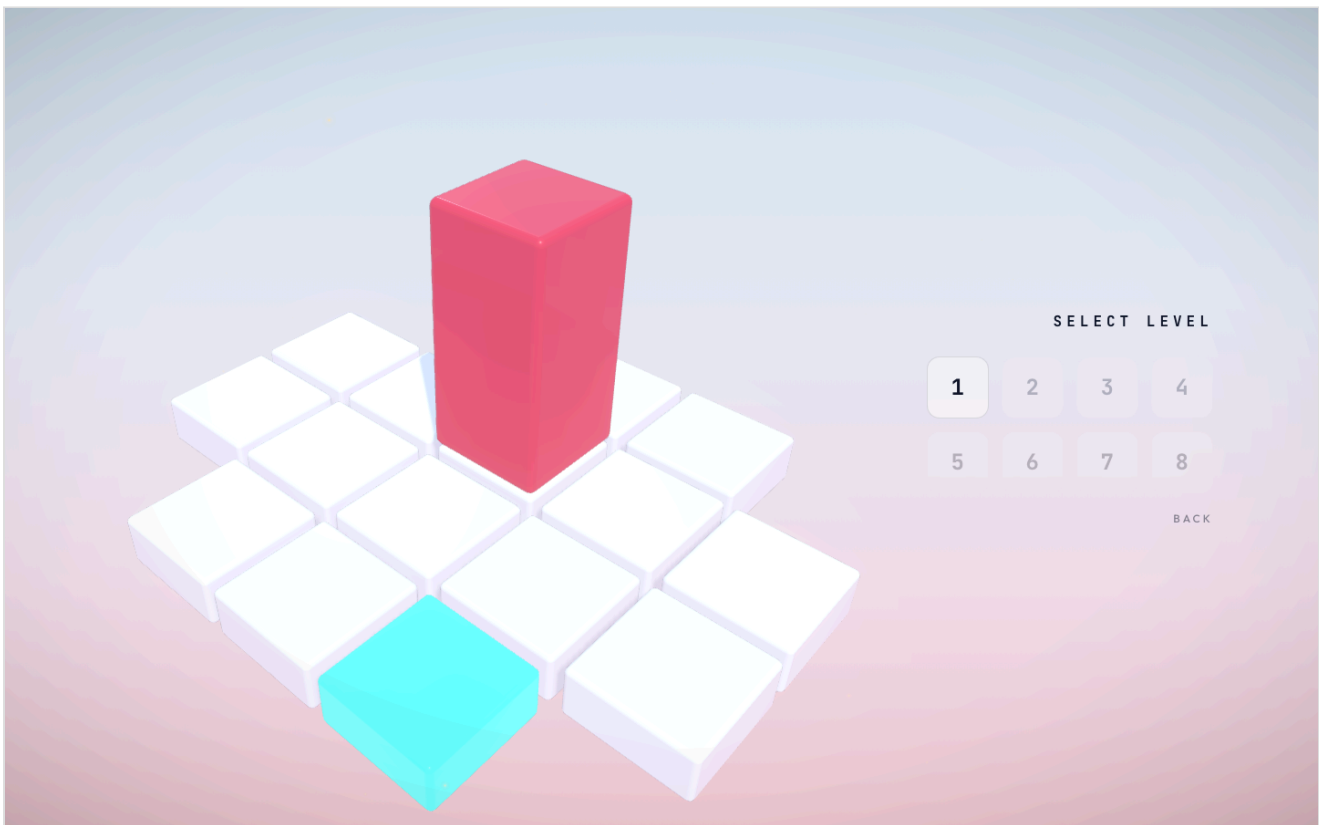
The final implementation includes:

- Eight official levels with verified shortest par values.
- A rolling cuboid with three logical orientations: `standing`, `lying_x`, and `lying_z`.
- Normal tiles, fragile tiles, soft switches, heavy switches, bridges, teleport split mode, cube merge, and goal validation.
- A custom level editor with raycast-based mouse placement.
- Level progression, completion state, and best move counts saved with `localStorage`.
- Keyboard controls, touch swipe controls, undo, restart, hints, and responsive UI.
- A smooth third-person camera for gameplay and separate menu/editor camera flows.
- WebGL rendering with physical materials, shadows, fog, procedural sky, environment lighting, SMAA, bloom, and vignette.
- Procedural sound effects through the Web Audio API.
- Dust, shatter, bridge, switch, sparkle, and win feedback.
- A BFS hint solver for non-teleport official levels.
- A verification script that checks the listed par values for all eight official levels, including the split-cube level.

Implementation Snapshots



Home screen with the 3D menu preview.



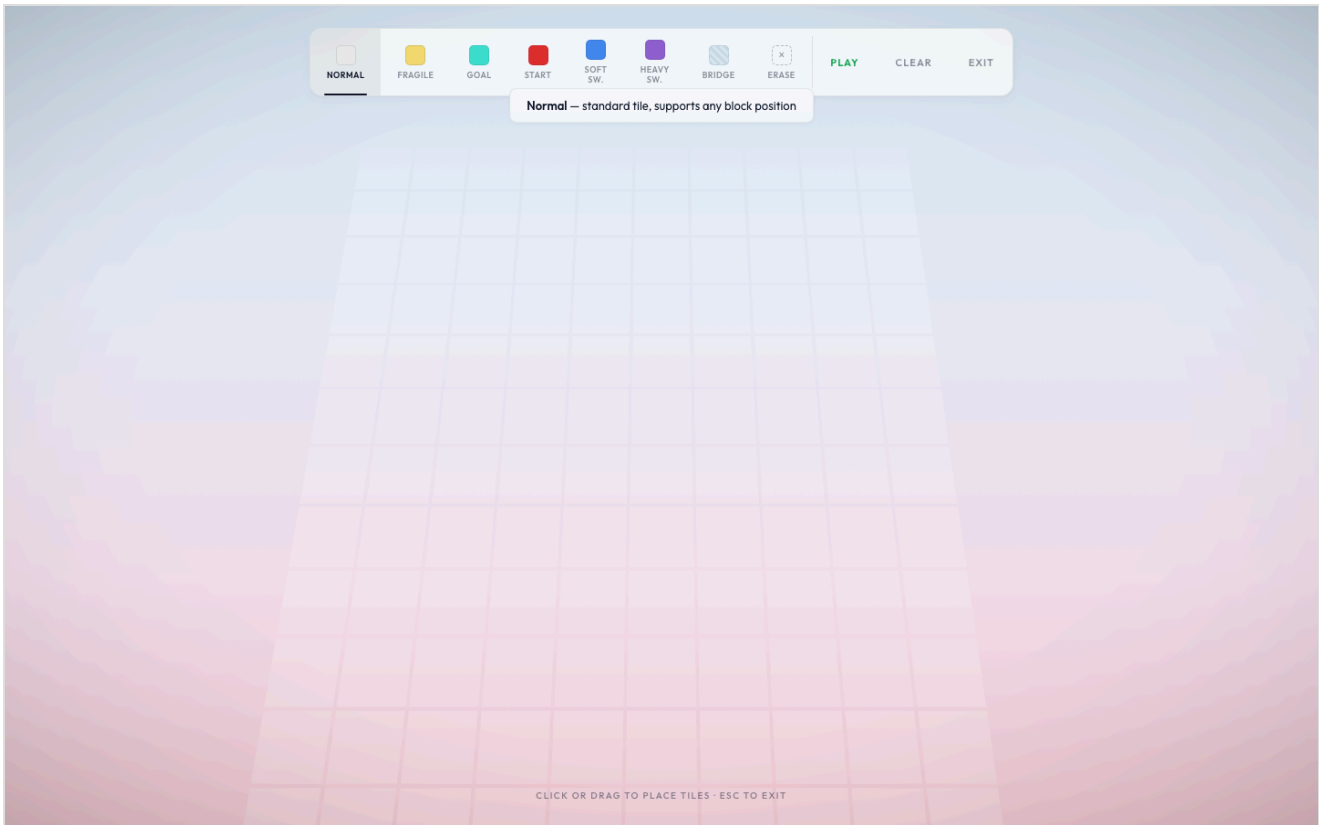
Level selection screen with progress-aware level buttons.



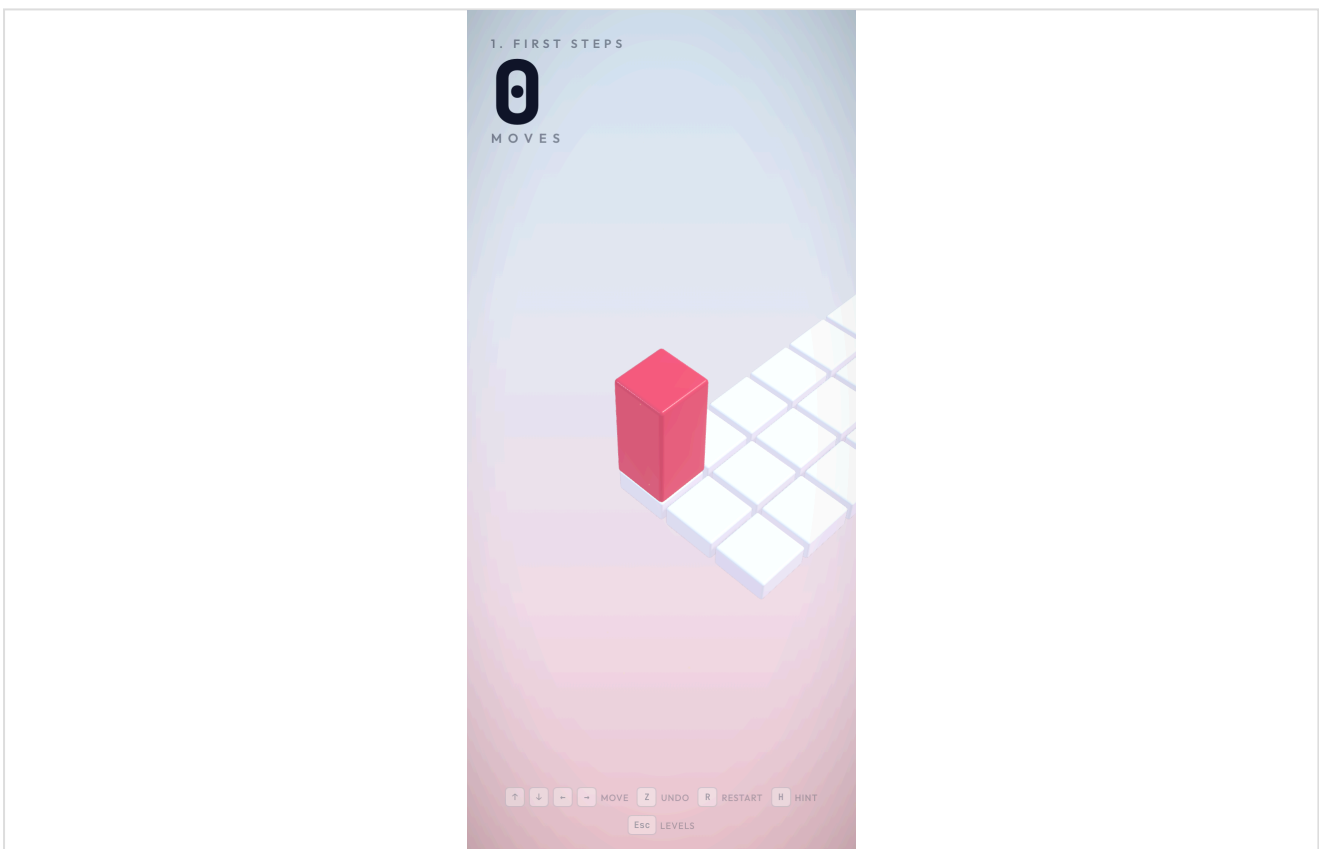
Gameplay view with the block, board, HUD, and goal tile.



Hint view after pressing H. The solver highlights the next occupied cells.



Editor view with the tool strip and raycast placement grid.



Mobile viewport test with the same gameplay scene and responsive HUD.

Project Structure

The code is split by responsibility:

- `src/main.js` controls the application flow: home screen, level selection, gameplay, editor entry, level loading, transitions, hints, and the render loop.
- `src/controls.js` implements movement, validation, falling, win behavior, undo, switches, teleport split mode, cube movement, and cube merging.
- `src/grid.js` builds the tile board, switch indicators, bridge tiles, bridge state, tile removal, and hint highlights.
- `src/block.js` creates the main block and split cubes.
- `src/levels.js` stores official levels, starting cells, par values, switches, bridges, and visual theme data.
- `src/solver.js` implements the BFS hint solver for normal block levels.
- `scripts/check-levels.js` verifies all official par values and includes a separate split-mode search for the teleport level.
- `src/editor.js` implements the custom level editor with raycasting.
- `src/scene.js`, `src/camera.js`, `src/lighting.js`, and `src/themes.js` set up the visual world.
- `src/particles.js`, `src/sounds.js`, and `src/shake.js` provide feedback.

Game Flow

The application has four main user-facing states.

Home is the starting state. It shows the title, a small 3D preview board, and buttons for Play and Editor.

Level select shows all eight official levels. Locked levels are dimmed. Completed levels are marked. If the saved best move count is less than or equal to the level par, the level also gets a star.

Gameplay shows the active board, the rolling block, move counter, level title, control hints, and temporary overlays for win/fall states.

Editor shows a top-down editable grid with tools for normal tile, fragile tile, goal, start, soft switch, heavy switch, bridge, and eraser. A custom level can be tested immediately and then edited again.

Progress is saved in `localStorage` under one project key. The saved data stores completed level IDs and best move counts.

Level Data and Board Representation

Each official level is stored as a 2D grid in `src/levels.js`. A number represents the tile type:

- 0: empty space
- 1: normal tile
- 2: goal tile
- 3: fragile tile
- 4: soft switch
- 5: heavy switch
- 6: teleport switch

Switches and bridges are stored separately from the basic layout. This is important because bridge tiles can appear and disappear while the level is running. The layout tells the game where the fixed tiles are; the bridge state tells the game which dynamic tiles currently exist.

The official levels are:

- Level 1, First Steps, par 8: normal movement.
- Level 2, Over the Edge, par 7: narrow paths and falling risk.
- Level 3, Fragile Ground, par 6: fragile tile behavior.
- Level 4, Open Sesame, par 6: soft switch and bridge.
- Level 5, Heavy Duty, par 12: heavy switch behavior.
- Level 6, Split Decision, par 8: teleport split mode.
- Level 7, Twin Bridges, par 13: combined soft and heavy bridge routes.
- Level 8, The Gauntlet, par 15: combined mechanics.

Puzzle State

The main block state is:

```
x, z, orientation
```

The orientation has three values:

```
standing  
lying_x  
lying_z
```

When the block is `standing`, it occupies one tile. When it is `lying_x`, it occupies two neighboring tiles along the x axis. When it is `lying_z`, it occupies two neighboring tiles along the z axis.

This state is used for all gameplay decisions. The mesh position is not guessed from physics. Instead, the code computes occupied cells from the exact state and checks those cells against the grid.

Examples:

- A standing block at $x = 2, z = 3$ occupies only row 3, column 2.
- A `lying_x` block centered at $x = 2.5, z = 3$ occupies row 3, columns 2 and 3.
- A `lying_z` block centered at $x = 2, z = 3.5$ occupies rows 3 and 4, column 2.

The occupied cells decide whether the block falls, whether a switch activates, whether a fragile tile breaks, and whether the goal is reached.

Movement Implementation

Movement starts from a keyboard event, touch swipe, or split-cube move. The code first computes the next logical state. Then it animates the mesh toward that state.

For the full block:

- From standing, a move makes the block lie down. The center moves 1.5 cells because the block goes from one occupied cell to two occupied cells.
- From `lying_x`, moving left or right can stand the block up. Moving up or down slides the lying block sideways by one cell.
- From `lying_z`, moving up or down can stand the block up. Moving left or right slides the lying block sideways by one cell.

The visual movement uses:

- `Vector3.LerpVectors` for smooth position interpolation.
- `Quaternion.SlerpQuaternions` for smooth rotation.
- A short vertical arc during rolling so the block feels like it pivots instead of sliding.
- A final snap to the exact target position and quaternion to prevent floating-point drift.

This is a direct use of transformation concepts from the course. The logical state changes in grid space, and the visible mesh changes with translation and rotation in world space.

Tile Mechanics

Normal tiles support the block in any legal orientation.

Goal tiles finish the level only when the full block is standing upright on them. A lying block touching the goal is not enough.

Fragile tiles break only when the full block stands upright on them. The tile is removed, shatter feedback plays, and the block falls.

Soft switches activate when any occupied cell touches them. A standing block, lying block, or split cube can trigger a soft switch.

Heavy switches activate only when the full block stands upright on them. This forces the player to plan orientation, not only position.

Bridges are dynamic support tiles. Switches toggle their target bridge cells. The bridge state is saved in move history and included in solver states.

Teleport switches split the full block into two smaller cubes. The player controls one cube at a time and changes the active cube with Space. If the two cubes become adjacent, they merge back into one lying block. The merge orientation depends on whether the two cubes are adjacent along x or along z.

Editor Implementation

The editor uses the same Three.js scene, but it switches the camera to a stable top-down view. It keeps an internal 14 by 10 grid and draws a faint placement grid.

Mouse placement uses `THREE.Raycaster`. The mouse position is converted to normalized device coordinates, a ray is cast from the camera, and the ray intersects a horizontal plane. The hit point is rounded to a row and column. That cell is then edited with the active tool.

The editor enforces one start tile and one goal tile. If a new goal is placed, the old goal is converted back to a normal tile. Custom bridge behavior is intentionally simple: custom switches target all custom bridge cells. This makes the editor usable during a short demo without needing a separate bridge-linking UI.

Rendering and Course Concepts

The project follows the graphics pipeline discussed in the lecture notes:

```
scene data
geometry
model transform
view transform
projection
rasterization
fragment shading
frame buffer
```

The level layout starts as data. `src/grid.js` converts that data into tile meshes. The block and cubes are also meshes. These objects are placed in world space through model transforms.

The view transform comes from the camera. In gameplay, the camera follows the active block or active split cube from a fixed offset. It lerps toward a target camera position and target look-at point so the framing stays smooth.

Projection is handled by `THREE.PerspectiveCamera`. This maps the 3D scene into the 2D browser canvas with perspective depth.

Rasterization is handled by WebGL through Three.js. Triangles from the block, tiles, bridge meshes, particles, and editor grid become fragments on the canvas.

Fragment shading is provided through Three.js materials and postprocessing effects. The project does not claim custom GLSL shaders for the block. Three.js and the postprocessing library generate and manage the shader programs used for materials and effects.

The course ray/path tracing lecture is still relevant conceptually, but the final project uses real-time rasterized rendering. That is the appropriate choice for an interactive puzzle game with constant input, animation, and UI.

Lighting, Materials, and Visual Feedback

The scene uses several light types:

- `AmbientLight` gives a base amount of light so shadowed areas are still readable.
- `HemisphereLight` adds a soft sky/ground color contribution.
- `DirectionalLight` acts like a sun and casts shadows.
- A second directional fill light softens the opposite side.

The main shadow-casting light follows the active block position. This keeps the shadow camera centered near the board and avoids wasting shadow resolution far away from gameplay.

Tiles and blocks use `MeshPhysicalMaterial`. Important material parameters include color, roughness, metalness, clearcoat, emissive color, and environment intensity. Goal tiles use emissive color to stand out. Switches have 3D indicator shapes on top.

The background sky is procedural. A canvas is drawn with a gradient, clouds, and soft mountain shapes, then used as a `CanvasTexture`. Themes change sky colors, fog color, tile color, goal color, block color, and sparkle color.

Postprocessing uses:

- SMAA to reduce jagged edges.
- Bloom to give bright goal and effect areas a soft glow.
- Vignette to slightly darken the edges and keep attention on the board.

Feedback systems are also implemented:

- Dust particles on landing.
- Shatter particles when a fragile tile breaks.

- Bridge and switch feedback when a switch changes the board.
- Celebration particles on win.
- Small screen shake on heavy events.
- Sparkles around the level using the [@pmndrs/vanilla](#) Sparkles helper.

Audio and Persistence

Audio is procedural and does not use external sound files. `src/sounds.js` creates tones and noise through the browser Web Audio API. Movement uses a short tone and noise burst, falling uses a descending oscillator and noise, and winning uses a small sequence of tones.

Persistence uses `localStorage`. The game saves completed levels and best move counts. This is why progress remains after refreshing the page.

These two browser APIs are used directly. The effects themselves are implemented in the project code.

BFS Hint Solver

The hint solver is one of the most important implementation parts.

BFS means breadth-first search. It explores a graph one layer at a time. In this project, each graph node is a possible puzzle state, and each edge is one legal move.

The solver starts from the current block state. It then tries all four movement directions. Every valid result becomes a new state in the queue. Then the solver tries all moves from those states, and so on.

Because BFS explores by distance from the start, the first time it reaches the goal, the solution is guaranteed to use the smallest possible number of moves. That is why the hint can use the first move of the BFS solution as a shortest-path hint.

The state key includes:

```
x position  
z position  
orientation  
bridge mask
```

The bridge mask is important. The same block position can mean different things depending on which bridges are currently visible. For example, standing at the same cell before and after a switch is not always the same puzzle state. A bridge may now exist, or it may have disappeared. The solver stores bridge states as bits so it can compare states quickly.

The solver also avoids infinite loops with a `visited` set. If the same position, orientation, and bridge state has already been explored, it is skipped.

The in-game hint solver intentionally returns `null` for teleport levels. Split mode has more state because it has two cubes, one active cube, and merge behavior. Instead of forcing that extra logic into the in-game hint UI, the project handles split-mode verification in `scripts/check-levels.js`.

Par Verification

Par means the expected best move count for a level. In this project, par is not guessed. It is checked by script.

`npm run check:levels` loads every official level and solves it. For normal levels, it uses `src/solver.js`. For the teleport split level, the script includes an expanded BFS that can represent two split cubes, the active cube index, bridge state, and merge back into a full block.

The script fails if:

- A level has no solution.
- The shortest solution length does not match the level's listed par.

Current verified values:

1	First Steps	8
2	Over the Edge	7
3	Fragile Ground	6
4	Open Sesame	6
5	Heavy Duty	12
6	Split Decision	8
7	Twin Bridges	13
8	The Gauntlet	15

This gives a practical quality check. If a level is edited badly or a movement rule breaks, the script catches it before the demo.

Testing and Verification

The following checks were run while preparing the final submission:

```
npm run check:levels
npm run build
npm audit
Playwright browser walkthrough and screenshot capture
```

The Playwright walkthrough opened the actual local app, cleared saved progress, captured the home screen, level select, gameplay, hint state, editor, and mobile viewport. The screenshot files used in this report come from that run.

What Was Implemented Directly and What Uses Libraries

Directly implemented in this project:

- Level data and custom level format.
- Block movement rules and orientation transitions.
- Occupied-cell validation.
- Fragile tile, soft switch, heavy switch, bridge, teleport split, merge, fall, and win logic.
- Undo and move history.
- BFS hint solver for normal/switch levels.
- Split-mode BFS verification in `scripts/check-levels.js`.
- Level editor state, custom level generation, and editor UI flow.
- Procedural audio patterns using Web Audio API nodes.
- Procedural sky drawing on a canvas.
- Game UI flow, saved progress format, and responsive layout.

External libraries and APIs used:

- Three.js: WebGL renderer, scene graph, camera, vectors, quaternions, geometries, materials, lights, shadows, fog, raycaster, texture helpers, and rounded box geometry addon.
- postprocessing: EffectComposer, render/effect passes, SMAA, Bloom, and Vignette.
- @pmndrs/vanilla: Sparkles helper for floating sparkle particles.
- Vite: development server and production build tooling.
- Browser Web Audio API: oscillator, gain, buffer, and filter nodes for procedural sound.
- Browser localStorage: saved progress and best move counts.

No outside Bloxorz source code was copied. The Bloxorz game idea is credited as inspiration, and the implementation here is written around this project's own level data, movement code, rendering setup, editor, and solver.

Limitations

The in-game hint solver does not currently provide hints for teleport split levels. The split level is still verified by script, but live split-mode hints would need a larger UI because the answer may include switching the active cube.

The editor intentionally uses a simple bridge model: every custom switch targets every custom bridge. This keeps the editor easy to explain, but a larger editor would need a bridge-linking mode.

The project uses library-managed shaders through Three.js and postprocessing. Custom GLSL shaders are not part of the current implementation.

The game uses rasterized WebGL rendering. It does not implement ray tracing/path tracing. That choice keeps the project interactive and stable for the live demo.

The production build passes, but Vite reports a chunk-size warning because Three.js, postprocessing, and the game code are bundled into one main client chunk. This does not stop the submission build. A future version could code-split the editor or visual effects if download size became a priority.

Conclusion

Bloxorz 3D combines exact puzzle logic with real-time 3D rendering. The strongest part of the implementation is the separation between discrete state and continuous graphics. The rules are checked from exact occupied cells, while the player sees smooth rolling, falling, splitting, merging, camera motion, lighting, shadows, particles, postprocessing, and audio feedback.

From a Computer Graphics point of view, the project demonstrates geometry creation, transformations, viewing, projection, rasterization, shading, lighting, shadows, texture use through a procedural sky, postprocessing, interaction, and frame-by-frame rendering. From a software point of view, it also includes persistence, editor tooling, BFS hints, and automated level verification.

The result is a complete playable project that can be demonstrated live and also explained from the implementation side.

References

- Course lecture notes in [LECTURE_Notes](#): graphics pipeline, OpenGL overview, transformations, viewing/projection, shading, shadows, and ray/path tracing.
- [Three.js documentation](#) for WebGL rendering, scene graph, cameras, lights, materials, vectors, quaternions, raycasting, and geometry helpers.
- [Three.js Raycaster documentation](#) for editor mouse-to-grid placement.
- [Three.js MeshPhysicalMaterial documentation](#) for physically based material parameters.
- [Vite guide](#) for the development server and production build tooling.
- [pmndrs postprocessing documentation](#) for EffectComposer, SMAA, Bloom, and Vignette effects.
- [pmndrs/drei-vanilla GitHub repository](#) for the Sparkles helper used for floating sparkle particles.
- [MDN Web Audio API](#) for procedural audio nodes.
- [MDN localStorage documentation](#) for browser-side persistence.
- [Coolmath Games Bloxorz page](#) for the original Bloxorz gameplay idea and control objective credited as inspiration.